

ODT Library

Layer Gap Analysis and Implementation

Line-by-Line Explanation of Every Layer File

Files: Layer.h/c Relu.h/c Softmax.h/c Flatten.h/c Dropout.h/c LayerNorm.h/c MaxPool1d.h/c AvgPool1d.h/c AdaptiveAvgPool1d.h/c QuantizationLayer.h/c Conv1d.h/c Conv1dTransposed.h/c

Each chapter: (1) what was missed in existing guides, (2) full gap implementation with technical concepts

Chapter 1 -- Layer.h / Layer.c: Dispatch Infrastructure

GAP IDENTIFIED

GAP: The existing guides explain individual layers but never cover Layer.h/Layer.c, which is the central dispatch table binding all layers together.

MISSING: layerType_t enum, layerConfig_t union, layerFunctions[] table with designated initializers, initLayer(), function pointer types.

1.1 -- Layer.h: Types and Structures

Layer.h defines the entire layer system. Every layer in the ODT library is accessed through a single layer_t struct containing a type tag and a config pointer.

typedef enum layerType {	<-- enum tag: which layer is this?
LINEAR, RELU, CONV1D, CONV1D_TRANSPOSED,	
MAXPOOL1D, AVGPPOOL1D, SOFTMAX, FLATTEN,	
QUANTIZATION, ADAPTIVE_AVGPPOOL1D, DROPOUT,	
LAYERNORM	
} layerType_t;	<-- 12 layer types total

CONCEPT: layerType_t -- Type Tag Pattern

Every layer_t carries a layerType_t tag (an integer enum).

This lets the dispatch table and free functions know which union branch is active.

C unions have no built-in type info -- the programmer must track it externally.

layerType_t IS that external type record. Without it, reading the wrong union field is undefined behavior (reads garbage bytes as a pointer).

typedef union layerConfig {	<-- union: all 11 config pointers share one slot
linearConfig_t *linear;	
reluConfig_t *relu;	
softmaxConfig_t *softmax;	
conv1dConfig_t *conv1d;	
conv1dTransposedConfig_t *conv1dTransposed;	
maxPool1dConfig_t *maxPool1d;	
avgPool1dConfig_t *avgPool1d;	
adaptiveAvgPool1dConfig_t *adaptiveAvgPool1d;	
dropoutConfig_t *dropout;	
layerNormConfig_t *layerNorm;	
quantizationConfig_t *quantization;	
} layerConfig_t;	<-- size = sizeof(largest member) = sizeof(void*)

CONCEPT: C Union Memory Layout

All union members share the SAME memory address.

sizeof(layerConfig_t) = sizeof(pointer) = 4 or 8 bytes (one pointer slot).

linearConfig_t *linear at offset 0

reluConfig_t *relu at offset 0 <- same address!

Reading the wrong member gives the same bits reinterpreted as a different type.

This is why layerType_t is always checked before accessing the union.

typedef struct layer {	<-- the central layer object
layerType_t type;	<-- WHICH config is active
layerConfig_t *config;	<-- pointer to the active config
} layer_t;	

1.2 -- Function Pointer Types

typedef void (*forwardFn_t)(layer_t *layer,	<-- function pointer type
tensor_t *inputTensor, tensor_t *outputTensor);	
typedef void (*backwardFn_t)(layer_t *layer,	
tensor_t *forwardInput, tensor_t *loss, tensor_t	
*propLoss);	
typedef void (*calcOutputShapeFn_t)(layer_t	
*layer,	
shape_t *inputShape, shape_t *outputShape);	

CONCEPT: Function Pointer Types -- Why Typedef?

void (*forwardFn_t)(layer_t*, tensor_t*, tensor_t*) declares a type name for a pointer to a function with that signature.

Without typedef you must write: void (*fn)(layer_t*, tensor_t*, tensor_t*);

With typedef you write: forwardFn_t fn;

Every layer implements the same three function signatures -- this is the interface contract.
layerFunctions_t bundles all three for one layer type.

typedef struct layerFunctions {	
forwardFn_t forward;	<-- pointer to the forward function
backwardFn_t backward;	<-- pointer to the backward function
calcOutputShapeFn_t calcOutputShape;	<-- pointer to shape calculator
} layerFunctions_t;	
extern layerFunctions_t layerFunctions[];	<-- declared here, defined in Layer.c

1.3 -- Layer.c: The Dispatch Table

Layer.c contains the most important data structure in the ODT library: the layerFunctions[] array. It maps every layerType_t to its three functions.

layerFunctions_t layerFunctions[] = {	<-- array of structs
[LINEAR] = {linearForward, linearBackward, linearCalcOutputShape},	<-- index=0
[RELU] = {reluForward, reluBackward, reluCalcOutputShape},	<-- index=1
[CONV1D] = {conv1dForward, conv1dBackward, conv1dCalcOutputShape},	<-- index=2
[SOFTMAX] = {softmaxForward, softmaxBackward, softmaxCalcOutputShape},	
[FLATTEN] = {flattenForward, flattenBackward, flattenCalcOutputShape},	
[DROPOUT] = {dropoutForward, dropoutBackward, dropoutCalcOutputShape},	
[LAYERNORM] = {layerNormForward, layerNormBackward, layerNormCalcOutputShape},	
/* ... 12 entries total ... */	
};	

CONCEPT: Designated Initializers in C99

[LINEAR] = {...} means: set element at index LINEAR (=0) to this value.

[RELU] = {...} means: set element at index RELU (=1) to this value.

This is order-independent -- you can list them in any order.

If a new layer is added to the enum with value 5, the existing [RELU]=[1] entry is unaffected. Without designated initializers, inserting a new enum value would shift all subsequent entries and break the mapping silently.

This is the same pattern used in TensorConversion.c's conversionMatrix[6][6].

CONCEPT: How the Dispatch Works at Runtime

To call forward on any layer:

```
layerFunctions[layer->type].forward(layer, input, output);
```

layer->type is an integer (e.g., RELU=1).

layerFunctions[1] is the RELU entry: {reluForward, reluBackward, reluCalcOutputShape}.

.forward is reluForward. So this becomes: reluForward(layer, input, output).

Cost: one array lookup ($O(1)$) + one indirect call through function pointer.

On Cortex-M7: ~3-4 clock cycles total for the dispatch overhead.

Compare to a 12-case switch: same cost but dispatch table is more extensible.

void initLayer(layer_t *layer, layerType_t type, layerConfig_t *config) {	
layer->type = type;	<-- store type tag
layer->config = config;	<-- store config pointer
}	

initLayer is trivial -- just stores two values. All the real setup happens in each layer's own init function before initLayer is called.

Chapter 2 -- Relu.h / Relu.c

GAP IDENTIFIED

GAP: Existing guides mention ReLU conceptually but never explain the actual implementation.

MISSING: gteFloatValue/gteSymInt32Zero dispatch, scale propagation in SYM_INT32, backward gate (zero out where input<=0), memcpy for output shape.

2.1 -- reluConfig_t

typedef struct reluConfig {	
quantization_t *forwardQ;	<-- qtype for forward pass
quantization_t *backwardQ;	<-- qtype for backward pass
bool ownsQuantizations;	<-- true: free tears down quant structs
} reluConfig_t;	
void reluInitConfig(reluConfig_t *cfg,	
quantization_t *fQ, quantization_t *bQ) {	
cfg->forwardQ = fQ;	<-- store pointer, no copy
cfg->backwardQ = bQ;	<-- store pointer, no copy
}	

2.2 -- reluForwardFloat

void reluForwardFloat(tensor_t *input, tensor_t *output) {	
gteFloatValue(input, 0, 0, output);	<-- element-wise: out[i] = in[i]>=0 ? in[i] : 0
}	

CONCEPT: gteFloatValue -- Greater Than or Equal Comparison

gteFloatValue(input, threshold, replaceValue, output) applies element-wise:

output[i] = (input[i] >= threshold) ? input[i] : replaceValue

For ReLU: threshold=0, replaceValue=0

output[i] = (input[i] >= 0) ? input[i] : 0

This is equivalent to: output[i] = max(0, input[i])

The comparison helper lives in Comparison.c and is reused across layers.

2.3 -- reluForwardSymInt32

void reluForwardSymInt32(tensor_t *input, tensor_t *output) {	
---	--

symInt32QConfig_t *inQC = input->quantization->qConfig;	<-- get scale
symInt32QConfig_t *outQC = output->quantization->qConfig;	<-- get scale
gteSymInt32Zero(input, 0, output);	<-- mantissa: out[i] = in[i]>=0 ? in[i] : 0
outQC->scale = inQC->scale;	<-- PROPAGATE scale unchanged
}	

CONCEPT: Scale Propagation Through ReLU

ReLU is a scale-transparent operation: it zeros some elements and passes others unchanged.

The scale does NOT change because:

- Inactive elements become 0 (mantissa 0, same as real 0 regardless of scale)
- Active elements pass through with the SAME mantissa value
- Therefore the scale mapping is unchanged: real = mantissa x scale

Compare to matmul which multiplies two scales together (output_scale = A_scale x B_scale).

ReLU: output_scale = input_scale. One assignment, no arithmetic.

2.4 -- reluBackwardFloat

ReLU backward implements the gate: gradient passes through where input>0, is zeroed where input<=0.

void reluBackwardFloat(tensor_t *forwardInput, tensor_t *loss, tensor_t *propLoss) {	
size_t N = calcNumberOfElementsByTensor(forwardInput);	
float *inputArray = (float *)forwardInput->data;	<-- original input from forward
float *gradOut = (float *)loss->data;	<-- gradient from next layer
float *gradIn = (float *)propLoss->data;	<-- gradient to previous layer
for (size_t i=0; i<N; i++) {	
if (inputArray[i] <= 0) {	<-- was pruned by ReLU?
gradIn[i] = 0;	<-- zero gradient: neuron was dead
} else {	
gradIn[i] = gradOut[i];	<-- pass gradient through unchanged
}	
}	
}	

CONCEPT: ReLU Backward: The Gradient Gate

ReLU(x) = max(0,x). Its derivative is: dReLU/dx = 1 if x>0, else 0.

Chain rule: dL/dx = dL/dy * dReLU/dx

= dL/dy * 1 if x>0 (pass gradient through)

= $dL/dy * 0$ if $x \leq 0$ (block gradient: dead neuron)

The forward input tensor (from the forward pass) is needed here to know which neurons were active ($input > 0$) vs dead ($input \leq 0$).

Dead neurons: $input \leq 0$ during forward $\rightarrow$ output was 0 $\rightarrow$ gradient is 0.

No information passed through, so no gradient flows back.

CONCEPT: Why forwardInput Is Saved Through the Forward Pass?

In backpropagation, the backward pass of each layer needs information from its forward pass.

For ReLU, we need to know WHICH inputs were positive (the gate mask).

The training loop must preserve forwardInput between forward and backward calls.

This is called the activation buffer or checkpoint buffer.

Memory cost: one float per neuron per layer kept alive between forward and backward.

For a 512-unit ReLU: $512 \times 4 = 2$ KB kept alive. For 10 layers: 20 KB.

On a 320 KB MCU this must be carefully budgeted.

2.5 -- reluBackwardSymInt32

<pre>void reluBackwardSymInt32(tensor_t *forwardInput, tensor_t *loss, tensor_t *propLoss) {</pre>	
<pre>int32_t *inputArray = (int32_t *)forwardInput->data;</pre>	
<pre>int32_t *gradOutputArr = (int32_t *)loss->data;</pre>	
<pre>int32_t *gradInputArr = (int32_t *)propLoss->data;</pre>	
<pre>for (size_t i=0; i<N; i++) {</pre>	
<pre>if (inputArray[i] <= 0) { gradInputArr[i] = 0; }</pre>	<-- dead neuron
<pre>else { gradInputArr[i] = gradOutputArr[i]; }</pre>	<-- pass through
<pre>}</pre>	
<pre>symInt32QConfig_t *lossQC = loss->quantization->qConfig;</pre>	
<pre>symInt32QConfig_t *propLossQC= propLoss->quantization->qConfig;</pre>	
<pre>propLossQC->scale = lossQC->scale;</pre>	<-- propagate scale unchanged
<pre>}</pre>	

The mantissa check `inputArray[i] <= 0` works correctly because `SYM_INT32` is symmetric around zero: mantissa=0 means real=0, negative mantissa means negative real.

2.6 -- reluCalcOutputShape

void reluCalcOutputShape(layer_t *reluLayer, shape_t *input, shape_t *output) {	
memcpy(output->dimensions, input->dimensions, input->numberOfDimensions * sizeof(size_t));	<-- copy all dim sizes
memcpy(output->orderOfDimensions, input->orderOfDimensions, input->numberOfDimensions * sizeof(size_t));	<-- copy order (transposition state)
output->numberOfDimensions = input->numberOfDimensions;	<-- copy rank
}	

CONCEPT: memcpy for Shape: Why Copy orderOfDimensions Too?

memcpy(dst, src, N*sizeof(size_t)) copies N size_t values: the dimension sizes.

ALSO copying orderOfDimensions is important: if the input tensor was transposed
(orderOfDimensions=[1,0]), the ReLU output should preserve that transposition state.

ReLU is element-wise -- it does not care about logical vs physical layout.

If we only copied dimensions but not orderOfDimensions, a transposed input would
produce a non-transposed output with the wrong logical interpretation.

Chapter 3 -- Softmax.h / Softmax.c

GAP IDENTIFIED

GAP: Softmax math was mentioned but the numerically stable implementation and the exact backward formula were never explained.

MISSING: subtract-max trick for numerical stability, commented-out naive version, Jacobian-free backward formula $s^*(dL - \text{dot}(s, dL))$, SYM_INT32 conversion path.

3.1 -- softmaxForwardFloat: Numerically Stable Implementation

Softmax: $\text{output}[i] = \exp(x[i]) / \sum_j(\exp(x[j]))$.

<pre>static void softmaxForwardFloat(tensor_t *input, tensor_t *output) {</pre>	
<pre> size_t n = calcNumberOfElementsByTensor(input);</pre>	
<pre> float *x = (float *)input->data;</pre>	<-- input array
<pre> float *y = (float *)output->data;</pre>	<-- output array
<pre> /* Pass 1: find max */</pre>	
<pre> float max = x[0];</pre>	<-- start with first element
<pre> for (size_t i=1; i<n; i++) { if (x[i]>max) max=x[i]; }</pre>	<-- scan for max
<pre> /* Pass 2: exp(x-max) and sum */</pre>	
<pre> float sum = 0.f;</pre>	
<pre> for (size_t i=0; i<n; i++) {</pre>	
<pre> float e = expf(x[i] - max);</pre>	<-- subtract max before exp!
<pre> y[i] = e; sum += e;</pre>	<-- store and accumulate
<pre> }</pre>	
<pre> /* Pass 3: normalize */</pre>	
<pre> for (size_t i=0; i<n; i++) { y[i] /= sum; }</pre>	<-- divide by sum
<pre>}</pre>	

CONCEPT: Numerical Stability: The Subtract-Max Trick

NAIVE: $\exp(x[i]) / \sum_j(\exp(x[j]))$

STABLE: $\exp(x[i]-\max) / \sum_j(\exp(x[j]-\max))$

These are mathematically identical: $\exp(x[i]-\max) = \exp(x[i]) / \exp(\max)$

$\exp(x[i]-\max) / \sum_j(\exp(x[j]-\max))$

$= (\exp(x[i])/\exp(\max)) / (\sum_j \exp(x[j])/\exp(\max))$

$= \exp(x[i]) / \sum_j \exp(x[j])$ <- same result

WHY stable? The largest $x[i]-\max = 0$. So the largest exp value is $\exp(0)=1$.

Without subtraction: if $x[i]=100$, $\exp(100)=2.7e43 \rightarrow$ IEEE 754 overflow (inf).

With subtraction: $\exp(100-100)=\exp(0)=1 \rightarrow$ no overflow.

Three passes: (1) find max, (2) compute exp and sum, (3) normalize.

Cannot combine passes 1 and 2: max must be known before any exp is computed.

WARNING: Commented-Out Naive Version

The file contains a commented-out naive softmaxForwardFloat that does NOT subtract max. It is kept as documentation of the instability. For small logits (e.g. $x < 10$) both versions agree, but for speaker ID logits that can reach > 50 , the naive version overflows. Always use the stable version.

3.2 -- softmaxForwardSymInt32

SYM_INT32 softmax converts inputs to float, computes softmax in float, then converts output back. Softmax is inherently a float operation (expf requires floating point).

static void softmaxForwardSymInt32(tensor_t *input, tensor_t *output) {	
size_t inputSize = calcNumberOfElementsByTensor(input);	
/* --- Build float input tensor on stack --- */	
tensor_t inputFloat; quantization_t inputFloatQ;	
initFloat32Quantization(&inputFloatQ);	
uint8_t inputFloatData[inputSize * sizeof(float)];	<-- VLA
setTensorValuesForConversion(inputFloatData, &inputFloatQ, input, &inputFloat);	
convertTensor(input, &inputFloat);	<-- SYM_INT32 -> FLOAT32
/* --- Same for output --- */	
uint8_t outputFloatData[inputSize * sizeof(float)];	<-- VLA
/* ... set up outputFloat ... */	
/* NOTE: uses NAIVE softmax (no max subtraction) in SYM_INT32 path */	
float sum=0; for (size_t i=0; i<inputSize; i++) sum+=expf(inArr[i]);	
for (size_t i=0; i<inputSize; i++) outArr[i]=expf(inArr[i])/sum;	<-- naive!
convertTensor(&outputFloat, output);	<-- FLOAT32 -> SYM_INT32
}	

WARNING: SYM_INT32 Softmax Uses Naive (Unstable) Version

The SYM_INT32 path uses the naive softmax (no subtract-max trick). This is a gap -- if the dequantized float values are large, this will overflow. The float path (stable) and SYM_INT32 path (naive) give different numerical results. For the speaker ID task, logits are typically in a safe range, but this should be fixed for production use.

3.3 -- softmaxBackwardFloat: Jacobian-Free Formula

The full Jacobian of softmax is an NxN matrix. The backward pass uses an efficient closed-form that avoids materializing it.

static void softmaxBackwardFloat(tensor_t *input,	
tensor_t *loss, tensor_t *propLoss) {	
size_t n = calcNumberOfElementsByTensor(input);	
float *s = (float *)input->data;	<-- s = softmax OUTPUT (not raw input!)
float *dLds = (float *)loss->data;	<-- gradient of loss w.r.t. softmax output
float *dLdx = (float *)propLoss->data;	<-- gradient to propagate back
/* dot = sum_j(s[j] * dLds[j]) */	
float dot=0.0f;	
for (size_t i=0; i<n; i++) { dot += s[i]*dLds[i];	<-- inner product s . dLds
}	
for (size_t i=0; i<n; i++) {	
dLdx[i] = s[i] * (dLds[i] - dot);	<-- Jacobian-free backward
}	
}	

CONCEPT: Softmax Backward: Jacobian-Free Derivation

Full Jacobian: $d(\text{softmax}_i)/d(x_j) = s_i(\delta_{ij} - s_j)$

where $\delta_{ij} = 1$ if $i=j$, else 0

$$dL/dx_i = \sum_j dL/ds_j * ds_j/dx_i$$

$$= \sum_j dL/ds_j * s_j(\delta_{ij} - s_i)$$

$$= dL/ds_i * s_i(1 - s_i) + \sum_{j \neq i} dL/ds_j * s_j(-s_i)$$

$$= s_i * dL/ds_i - s_i * \sum_j (s_j * dL/ds_j)$$

$$= s_i * (dL/ds_i - \text{dot}(s, dL/ds))$$

In code: $dLdx[i] = s[i] * (dLds[i] - \text{dot})$

Cost: ONE dot product ($O(N)$) + ONE pass ($O(N)$) = $O(N)$ total.

Full Jacobian approach: $O(N^2)$ -- infeasible for large N.

CONCEPT: IMPORTANT: input here is Softmax OUTPUT, not Raw Input

The backward function parameter is named input but it receives the FORWARD OUTPUT (softmax probabilities), not the raw logits.

This is because the backward formula $s*(dL - \text{dot}(s,dL))$ uses $s=\text{softmax}(x)$, NOT x . The softmax output is what the training loop must save.

Contrast with ReLU backward which needs the raw input x (to check sign).

Each layer needs DIFFERENT saved tensors from its forward pass.

3.4 -- softmaxCalcOutputShape

void softmaxCalcOutputShape(layer_t *l, shape_t *input, shape_t *output) {	
memcpy(output->dimensions, input->dimensions,	
input->numberOfDimensions * sizeof(size_t));	<-- same shape
memcpy(output->orderOfDimensions,	
input->orderOfDimensions,	
input->numberOfDimensions * sizeof(size_t));	<-- preserve transposition
output->numberOfDimensions =	
input->numberOfDimensions;	
}	

Softmax is shape-preserving: output has the same dimensions as input.

Chapter 4 -- Flatten.h / Flatten.c

GAP IDENTIFIED

GAP: Flatten was treated as a trivial reshape but the implementation has subtle decisions around SYM_INT32 scale handling and shape broadcasting.

MISSING: why memcpy is valid here, scale copying for quantized tensors, how flattenCalcOutputShape computes the collapsed dimension.

4.1 -- flattenForward

Flattening collapses all dimensions after the batch dimension into one.

static void flattenForward(tensor_t *input, tensor_t *output) {	
size_t n = calcNumberOfElementsByTensor(input);	
memcpy(output->data, input->data, n * input->bytesPerElement);	<-- copy ALL bytes
if (input->quantizationConfig->type == SYM_INT32) {	
getQConfigOfTensor(output)->scale =	
getQConfigOfTensor(input)->scale;	<-- copy scale!
}	
}	

CONCEPT: Why memcpy Is Valid for Flattening

Flattening is a LOGICAL operation: it changes the shape metadata but leaves the underlying data bytes identical.

For FLOAT32: each float occupies 4 bytes, stored in row-major order. After flatten, the byte sequence is unchanged -- only the shape changes.

For SYM_INT32: each value occupies 4 bytes (int32_t). Same data layout. BUT: the quantization scale must be copied too, because the output tensor has its own quantizationConfig. Without the copy, output->scale = 0 -> all dequantized values become 0 (silent corruption).

IMPORTANT: orderOfDimensions is NOT copied here. Flatten always produces a contiguous [batch, flat] output with identity order. If the input was transposed (non-identity order), the bytes were already stored in the transposed layout -- memcpy copies them as-is, which is correct.

4.2 -- flattenBackward

Backward pass: gradient flows back unchanged.

static void flattenBackward(tensor_t *input, tensor_t *loss, tensor_t *propLoss) {	
size_t n = calcNumberOfElementsByTensor(loss);	

memcpy(propLoss->data, loss->data, n * loss->bytesPerElement);	<-- reshape loss back
if (loss->quantizationConfig->type == SYM_INT32) {	
getQConfigOfTensor(propLoss)->scale = getQConfigOfTensor(loss)->scale;	
}	
}	

WARNING: Flatten Backward: Identity Gradient

Flatten just reshapes -- it applies no transformation to values. Therefore $dL/d(input_i) = dL/d(output_i)$ for every i . The propLoss is the loss reshaped back to the input shape (memcpy + shape metadata).

4.3 -- flattenCalcOutputShape

void flattenCalcOutputShape(layer_t *l, shape_t *input, shape_t *output) {	
output->numberOfDimensions = 2;	<-- always [batch, flat]
output->dimensions[0] = input->dimensions[0];	<-- batch size unchanged
size_t flat = 1;	
for (size_t d=1; d<input->numberOfDimensions; d++) {	
flat *= input->dimensions[d];	<-- multiply all remaining dims
}	
output->dimensions[1] = flat;	
output->orderOfDimensions[0] = 0;	<-- identity order
output->orderOfDimensions[1] = 1;	
}	

CONCEPT: Shape Example

Input: [batch=8, channels=16, length=32] (3D)

flat = $16 * 32 = 512$

Output: [batch=8, flat=512] (2D)

For a speaker ID model: after Conv1d layers produce [8, 64, 128],

Flatten gives [8, 8192] which is then fed to Linear for classification.

Chapter 5 -- Dropout.h / Dropout.c

GAP IDENTIFIED

GAP: Dropout appeared trivial but has three non-obvious design decisions: (1) the scale factor $1/(1-p)$, (2) `SYM_INT32` scale-folding instead of value scaling, (3) `training=false` default in `initDropoutConfig`.

MISSING: why inverted dropout, exact mask storage, `SYM_INT32` scale-fold math, why training is false by default.

5.1 -- initDropoutConfig

<code>dropoutConfig_t *initDropoutConfig(</code>	
<code>float p, BOOL *mask, size_t maskSize) {</code>	
<code>if (p<0.f p>=1.f) { /* error */ return NULL; }</code>	<-- p in [0,1)
<code>if (mask==NULL) { /* error */ return NULL; }</code>	<-- mask required
<code>dropoutConfig_t *c =</code> <code>reserveMemory(sizeof(dropoutConfig_t));</code>	
<code>c->p = p;</code>	<-- drop probability
<code>c->mask = mask;</code>	<-- caller-owned BOOL array
<code>c->maskSize = maskSize;</code>	
<code>c->training = false;</code>	<-- IMPORTANT: default is eval mode
<code>return c;</code>	
<code>}</code>	

CONCEPT: Why training=false Is the Safe Default

If training were true by default and the caller forgot to set it to false for inference, the model would randomly zero activations during prediction -- catastrophic silent failure.

With `training=false` as default: inference works correctly without any configuration. Training code must explicitly set `c->training=true` before the training loop and false again before evaluation.

This follows the principle of safe defaults: the mode that cannot cause silent accuracy degradation is the default.

5.2 -- dropoutScale and Inverted Dropout

<code>static float dropoutScale(float p) {</code>	
<code>return 1.f / (1.f - p);</code>	<-- inverted dropout scale factor
<code>}</code>	

CONCEPT: Inverted Dropout: Why $1/(1-p)$?

During training with `dropout(p=0.5)`:

- Each unit is kept with probability 0.5
- Expected activation = $0.5 * x$

During inference (p=0): full network, expected activation = x.

PROBLEM: the expected magnitude is 0.5*x in training vs x in inference.

This scale mismatch corrupts inference.

SOLUTION: Inverted dropout scales kept units by $1/(1-p)$ during training:

Expected activation = $p * 0 + (1-p) * x * 1/(1-p) = x$

Same expected magnitude in training and inference!

At inference: no scaling needed (pass-through). This is why the eval path in forward is just: output = input.

5.3 -- dropoutForwardFloat

static void dropoutForwardFloat(dropoutConfig_t *c, tensor_t *in, tensor_t *out) {	
size_t n = calcNumberOfElementsByTensor(in);	
float *inArr = (float *)in->data;	
float *outArr = (float *)out->data;	
if (!c->training) {	<-- eval mode
memcpy(out->data, in->data, n*in->bytesPerElement);	<-- identity
return;	
}	
float scale = dropoutScale(c->p);	
for (size_t i=0; i<n; i++) {	
if (tensorBoolGet(c->mask, i)) {	<-- mask[i] == keep
outArr[i] = inArr[i] * scale;	<-- scale kept units
} else {	
outArr[i] = 0.f;	<-- zero dropped units
}	
}	
}	

void dropoutForward(layer_t *l, tensor_t *in, tensor_t *out) {	
dropoutConfig_t *c = (dropoutConfig_t *)l->config;	
bernoulliFillMask(c->mask, c->maskSize, 1.f - c->p);	<-- fill mask FIRST
if (out->quantizationConfig->type == FLOAT32)	
dropoutForwardFloat(c, in, out);	
else	
dropoutForwardSymInt32(c, in, out);	
}	

WARNING: bernoulliFillMask: Called Once Per Forward

bernoulliFillMask(mask, size, keep_prob) fills the BOOL array with a Bernoulli sample: each bit = 1 with probability (1-p), else 0. It is called ONCE at the start of each forward pass. The same mask is then used in backward to gate gradients identically to how activations were gated. This is critical: the mask must be consistent between forward and backward within the same mini-batch.

5.4 -- dropoutForwardSymInt32: Scale-Folding

For SYM_INT32, scaling the integer values by $1/(1-p)$ would require multiplying every element. Instead, the scale is folded into the quantization config.

static void dropoutForwardSymInt32(dropoutConfig_t	
*c, tensor_t *in, tensor_t *out) {	
size_t n = calcNumberOfElementsByTensor(in);	
int32_t *inArr = (int32_t *)in->data;	
int32_t *outArr = (int32_t *)out->data;	
quantization_t *inQC = getQConfigOfTensor(in);	
quantization_t *outQC = getQConfigOfTensor(out);	
float scale = dropoutScale(c->p);	<-- $1/(1-p)$
for (size_t i=0; i<n; i++) {	
if (tensorBoolGet(c->mask, i)) {	
outArr[i] = inArr[i];	<-- copy integer AS-IS
} else {	
outArr[i] = 0;	
}	
}	
outQC->scale = inQC->scale * scale;	<-- fold scale into qconfig!
}	

CONCEPT: Scale-Folding Math

SYM_INT32 represents: $\text{real} = \text{mantissa} * q_scale$

Naive approach: $\text{outArr}[i] = \text{inArr}[i] * (1/(1-p))$ [integer multiply]

Problem: integer * float = float, requires conversion

Also: multiply every element even when mask is all-keep

Scale-fold approach: $\text{outArr}[i] = \text{inArr}[i]$ [integer copy, no math]

$\text{outQC->scale} = \text{inQC->scale} * (1/(1-p))$

Real interpretation: $\text{outArr}[i] * \text{outQC->scale}$

$= \text{inArr}[i] * \text{inQC->scale} * (1/(1-p))$

$= \text{real_value} * (1/(1-p))$ <- correct!

Cost: ONE float multiply per forward pass (not per element).

For n=8192 elements: ~8192x cheaper than element-wise approach.

5.5 -- dropoutBackward

static void dropoutBackwardFloat(dropoutConfig_t	
*c, tensor_t *loss, tensor_t *propLoss) {	
size_t n = calcNumberOfElementsByTensor(loss);	
float scale = dropoutScale(c->p);	
for (size_t i=0; i<n; i++) {	
if (tensorBoolGet(c->mask, i)) {	<-- same mask as forward!
propLoss[i] = loss[i] * scale;	<-- pass gradient
} else {	
propLoss[i] = 0.f;	<-- zero gradient for dropped units
}	
}	
}	

WARNING: Dropout Backward: Straight-Through with Gating

Dropped units have zero output, so their contribution to the loss is zero. Therefore their gradient must also be zero (chain rule: $dL/dx_i = 0$ if $output_i = 0$). Kept units are scaled by $1/(1-p)$, so their gradient is also scaled by $1/(1-p)$. The mask from the forward pass is reused to gate gradients.

Chapter 6 -- LayerNorm.h / LayerNorm.c

GAP IDENTIFIED

GAP: LayerNorm groups, two-pass statistics, and the eps-inside-sqrt placement were not explained. The physOffset helper and its role in multi-dim access was missing.

MISSING: layerNormGroupSizes math, layerNormPhysOffset, two-pass stats (biased variance /N), eps inside sqrt (not outside), gamma/beta affine transform.

6.1 -- initLayerNormConfig

layerNormConfig_t *initLayerNormConfig(size_t numGroups, parameter_t *gamma, parameter_t *beta, float eps, BOOL ownsQuantizations) { /* gamma: scale parameter, beta: shift parameter *\	
layerNormConfig_t *c = reserveMemory(sizeof(layerNormConfig_t)); c->numGroups = numGroups; c->gamma = gamma; c->beta = beta; c->eps = eps; c->ownsQuantizations = ownsQuantizations; return c; }	<-- numerical stability

6.2 -- layerNormGroupSizes: Dividing Dimensions into Groups

LayerNorm normalizes over a group of elements. Groups partition the non-batch dimensions.

static void layerNormGroupSizes(layerNormConfig_t *c, shape_t *s, size_t *G, size_t *N) { *G = c->numGroups; *N = 1; for (size_t d=1; d<s->numberOfDimensions; d++) { *N *= s->dimensions[d]; } *N /= *G; }	<-- number of groups <-- skip batch dim <-- product of all non-batch dims <-- elements per group
---	---

CONCEPT: Group Sizes Example

Input shape: [batch=4, channels=64, length=32]

Non-batch dimensions: 64 x 32 = 2048 elements

numGroups = 8

$N = 2048 / 8 = 256$ elements per group

LayerNorm computes mean and variance over each group of 256 elements.

numGroups=1: Standard LayerNorm (normalize over all non-batch dims).

numGroups=channels: Group Norm reduces to Instance Norm.

numGroups=N_elements: Group Norm reduces to LayerNorm (each element group).

6.3 -- layerNormPhysOffset: Multi-Dim Index Mapping

physOffset maps (batch_idx, group_idx, element_within_group) to a flat byte offset in the data buffer.

static size_t layerNormPhysOffset(tensor_t *t, size_t batchIdx, size_t groupIdx, size_t elemInGroup) { size_t G, N; layerNormGroupSizes(/* config */ ..., &G, &N); size_t flatIdx = batchIdx * G * N /* batch stride */ + groupIdx * N /* group stride */ + elemInGroup; /* element within group */ /* use calcElementIndexByIndices to handle transposition */ size_t indices[t->numberOfDimensions]; unflattenIndex(flatIdx, t->shape, indices); return calcElementIndexByIndices(t, indices) * t->bytesPerElement; }	
	<-- flat -> multi-dim

WARNING: Why calcElementIndexByIndices (Not Simple Arithmetic)

If the tensor has a non-identity orderOfDimensions (zero-copy transpose), a simple flat index would access the wrong byte. calcElementIndexByIndices applies the dimension permutation first, so the physical byte address is always correct regardless of logical vs physical layout.

6.4 -- layerNormGroupStats: Two-Pass Statistics

Computes mean and variance for each (batch, group) pair.

static void layerNormGroupStats(tensor_t *t, size_t batchIdx, size_t groupIdx, float *mean, float *var) { size_t G, N; layerNormGroupSizes(..., &G, &N); /* Pass 1: mean */ float sum=0.f; for (size_t e=0; e<N; e++) { uint8_t *ptr = data + layerNormPhysOffset(t, batchIdx, groupIdx, e);	
---	--

sum += *(float *)ptr;	
}	
*mean = sum / N;	
/* Pass 2: biased variance */	
float varSum=0.f;	
for (size_t e=0; e<N; e++) {	
uint8_t *ptr = data + layerNormPhysOffset(t, batchIdx, groupIdx, e);	
float diff = *(float *)ptr - *mean;	
varSum += diff*diff;	
}	
*var = varSum / N;	<-- biased: divide by N (not N-1)
}	

CONCEPT: Two-Pass Stats and Biased Variance

Pass 1 (mean): Welford online algorithm could do this in one pass but accumulates floating-point error. Two-pass is more numerically stable.

Pass 2 (variance): Uses the already-computed mean. This avoids the numerically unstable one-pass formula $\text{Var}(X) = E[X^2] - E[X]^2$, which suffers catastrophic cancellation for large values.

Biased variance (divide by N, not N-1): LayerNorm uses population variance (biased). The difference is negligible for typical $N > 64$ and matches PyTorch's LayerNorm implementation.

6.5 -- layerNormForwardFloat: eps Inside sqrt

for (size_t e=0; e<N; e++) {	
float x = *(float *)(data + layerNormPhysOffset(t, b, g, e));	
float xhat = (x - mean) / sqrtf(var + eps);	<-- eps INSIDE sqrt
float y = gamma[g] * xhat + beta[g];	<-- affine transform
*(float *)(outData + layerNormPhysOffset(out, b, g, e)) = y;	
}	

CONCEPT: eps Inside sqrt (Not Outside)

CORRECT: $(x - \text{mean}) / \sqrt{\text{var} + \text{eps}}$

WRONG: $(x - \text{mean}) / (\sqrt{\text{var}} + \text{eps})$

When var is very small (near-constant input), $\sqrt{\text{var}} \rightarrow 0$.

WRONG formula: denominator = $\sqrt{\sim 0} + \text{eps} \approx \text{eps}$. Sensitive to eps.

CORRECT formula: denominator = $\sqrt{\sim 0 + \text{eps}} = \sqrt{\text{eps}}$. More stable.

Example: var=1e-10, eps=1e-5

CORRECT: $\sqrt{1e-10 + 1e-5} = \sqrt{1.000001e-5} \approx 0.00316$

WRONG: $\sqrt{1e-10} + 1e-5 = 1e-5 + 1e-5 = 2e-5$

PyTorch, TensorFlow, and the ODT library all use eps INSIDE sqrt.

6.6 -- LayerNorm Backward

Backward computes gradients w.r.t. x, gamma, beta.

<pre>/* dL/dbeta[g] = sum_e dLdy[b,g,e] */</pre>	
<pre>/* dL/dgamma[g] = sum_e dLdy[b,g,e] * xhat[b,g,e] */</pre>	
<pre>/* dL/dxhat[e] = dLdy[e] * gamma[g] */</pre>	
<pre>/* dL/dx[e] = (dLdxhat[e] - mean(dLdxhat) - xhat[e]*mean(dLdxhat*xhat)) / std */</pre>	

WARNING: LayerNorm Backward Complexity

The LayerNorm backward is more complex than most layers because the normalization creates dependencies between all elements in a group: changing one $x[i]$ changes the mean and variance, which affects all normalized values. The full backward must account for these cross-element dependencies, resulting in the three-term formula above.

Chapter 7 -- MaxPool1d.h / MaxPool1d.c

GAP IDENTIFIED

GAP: MaxPool1d argmax storage and the scatter-based backward were not explained.

MISSING: windowSlice1d_t struct, argmaxArr purpose, -INFINITY init, backward scatter (only argmax position gets gradient, all others zero).

7.1 -- initMaxPool1dConfig

maxPool1dConfig_t *initMaxPool1dConfig(size_t kernelSize, size_t stride, size_t padding, size_t *argmaxArr, size_t argmaxArrSize) { maxPool1dConfig_t *c = reserveMemory(sizeof(maxPool1dConfig_t)); c->kernelSize = kernelSize; c->stride = stride; c->padding = padding; c->argmaxArr = argmaxArr; c->argmaxArrSize = argmaxArrSize; return c; }	
	<-- caller-allocated, stores argmax indices

CONCEPT: argmaxArr: Why It Must Be Caller-Allocated

argmaxArr stores, for each output position, which INPUT position achieved the maximum. Size = output_length * batch * channels.

It cannot be heap-allocated inside initMaxPool1dConfig because:

1. The output shape is not yet known at config-init time.
2. Arena allocator (reserveMemory) has no free/realloc.

The caller computes the output shape first (maxPool1dCalcOutputShape), then allocates argmaxArr of the right size, then calls initMaxPool1dConfig.

Size formula: $\text{argmaxArrSize} = \text{batch} * \text{channels} * \text{outputLength}$

where $\text{outputLength} = \text{floor}((\text{inputLength} + 2 * \text{padding} - \text{kernelSize}) / \text{stride}) + 1$

7.2 -- maxPool1dForwardFloat: Argmax Tracking

static void maxPool1dForwardFloat(maxPool1dConfig_t *c, tensor_t *in, tensor_t *out) {	
--	--

/* for each batch b, channel ch, output position outPos: */	
windowSlice1d_t w = windowSlice1dAt(outPos, c->stride, c->padding, c->kernelSize, inputLen);	
float bestVal = -INFINITY;	<-- init to -inf
size_t bestInputIdx = w.start;	
for (size_t k=w.start; k<w.end; k++) {	
float v = inArr[b*chStride + ch*lenStride + k];	
if (v > bestVal) { bestVal=v; bestInputIdx=k; }	<-- track argmax
}	
outArr[b*outChStride + ch*outLenStride + outPos] = bestVal;	
c->argmaxArr[b*argChStride + ch*argLenStride + outPos] = bestInputIdx;	<-- SAVE argmax
}	

CONCEPT: Why -INFINITY Initialization

MaxPool must find the maximum value in a window.

If we initialize bestVal=0: a window of all-negative values would incorrectly return 0 (which is not in the window).

If we initialize bestVal=inArr[w.start]: valid but requires a special case for the first element.

-INFINITY (IEEE 754 negative infinity): any real float > -INFINITY, so the first comparison always updates bestVal. No special cases needed.

This is the standard idiom for running-max algorithms.

7.3 -- maxPool1dBackwardFloat: Scatter Gradient

Only the position that achieved the max (argmax) receives the gradient. All other positions in the window receive zero gradient.

static void maxPool1dBackwardFloat(maxPool1dConfig_t *c, tensor_t *loss, tensor_t *propLoss) {	
/* zero-initialize propLoss */	
memset(propLoss->data, 0, totalBytes);	<-- all zeros first
/* scatter: for each output position, route grad to argmax */	
for (b ...) for (ch ...) for (outPos ...) {	
size_t argmax = c->argmaxArr[b*S + ch*T + outPos];	<-- recall argmax
float lossGrad = lossArr[b*S2 + ch*T2 + outPos];	
propLossArr[b*S3 + ch*T3 + argmax] += lossGrad;	<-- +=, not =, handles ties
}	
}	

CONCEPT: MaxPool Backward: Scatter vs Gather

FORWARD (Gather): for each output, gather max from input window.

BACKWARD (Scatter): for each output gradient, scatter it back to argmax.

The += (not =) is important for overlapping windows ($\text{stride} < \text{kernelSize}$).

If two output windows both select the same input position as their argmax, that input receives the SUM of both output gradients.

Non-argmax positions receive gradient 0. Intuitively: they did not contribute to the output (the max was elsewhere), so they should not receive any update signal.

WHY argmax matters: If we did not store argmax during forward, we would need to recompute it in backward (recompute the entire forward pass). Storing argmax trades memory for compute -- the standard approach.

Chapter 8 -- AvgPool1d.h / AvgPool1d.c

GAP IDENTIFIED

GAP: AvgPool1d backward gradient distribution and count_include_pad behavior were unexplained.

MISSING: divisor calculation, count_include_pad=K always (no boundary correction), backward uniform distribution of gradient across all window positions.

8.1 -- avgPool1dForwardFloat

static void	
avgPool1dForwardFloat(avgPool1dConfig_t *c,	
tensor_t *in, tensor_t *out) {	
for (b ...) for (ch ...) for (outPos ...) {	
windowSlice1d_t w = windowSlice1dAt(outPos,	
c->stride, c->padding, c->kernelSize, inputLen);	
float sum = 0.f;	
size_t divisor = c->kernelSize;	<-- always K (count_include_pad)
for (size_t k=w.start; k<w.end; k++) {	
sum += inArr[b*S + ch*T + k];	
}	
outArr[outIdx] = sum / (float)divisor;	
}	
}	

CONCEPT: count_include_pad: Always K

At the boundaries with padding, the window may include padded positions.

The padded positions have value 0 (zero-padding).

count_include_pad=true: divisor = K always (padded zeros count)

count_include_pad=false: divisor = actual non-padded elements in window

The ODT library uses count_include_pad=true (divisor=K always).

This matches PyTorch AvgPool1d default (count_include_pad=True).

Effect at boundaries: output values near edges are slightly smaller because padded zeros pull down the average. This is the common choice for CNNs; it avoids boundary artifacts from varying divisors.

8.2 -- avgPool1dBackwardFloat: Uniform Distribution

Unlike MaxPool (which scatters to one position), AvgPool distributes the gradient equally to ALL positions in the window.

static void	
avgPool1dBackwardFloat(avgPool1dConfig_t *c,	

tensor_t *loss, tensor_t *propLoss) {	
memset(propLoss->data, 0, totalBytes);	<-- zero init
for (b ...) for (ch ...) for (outPos ...) {	
windowSlice1d_t w = windowSlice1dAt(...);	
float lossGrad = lossArr[outIdx];	
float distributed = lossGrad / (float)c->kernelSize;	<-- split evenly
for (size_t k=w.start; k<w.end; k++) {	
propLossArr[b*S+ch*T+k] += distributed;	<-- +=: overlap ok
}	
}	
}	

WARNING: AvgPool vs MaxPool Backward

MaxPool backward: ONE position (argmax) gets all gradient. AvgPool backward: ALL K positions get gradient/K. Both use += to correctly handle overlapping windows (stride < kernelSize). AvgPool does not need to store any intermediate values during forward (no argmax array needed), making it more memory-efficient.

Chapter 9 -- AdaptiveAvgPool1d.h / AdaptiveAvgPool1d.c

GAP IDENTIFIED

GAP: The adaptive window computation -- how output size maps to variable input windows -- was never explained.

MISSING: adaptiveWindow1dAt formula, why window sizes can differ between positions, backward distribution using w.count.

9.1 -- adaptiveWindow1dAt: Variable-Size Windows

Adaptive pooling maps a fixed OUTPUT size to a VARIABLE INPUT size by computing a different window for each output position.

windowSlice1d_t adaptiveWindow1dAt(size_t outIdx, size_t inputLen, size_t outputLen) {	
windowSlice1d_t w;	
w.start = (outIdx * inputLen) / outputLen;	<-- floor division
w.end = ((outIdx+1) * inputLen + outputLen-1) / outputLen;	<-- ceil division
w.count = w.end - w.start;	
return w;	
}	

CONCEPT: Adaptive Window Formula

For input length L_{in} and output length L_{out} :

$$\text{start}[i] = \text{floor}(i * L_{in} / L_{out})$$
$$\text{end}[i] = \text{ceil}((i+1) * L_{in} / L_{out})$$
$$\text{count}[i] = \text{end}[i] - \text{start}[i]$$

Example: $L_{in}=7$, $L_{out}=3$

$i=0$: $\text{start}=\text{floor}(0)=0$, $\text{end}=\text{ceil}(7/3)=\text{ceil}(2.33)=3$, $\text{count}=3$

$i=1$: $\text{start}=\text{floor}(7/3)=\text{floor}(2.33)=2$, $\text{end}=\text{ceil}(14/3)=\text{ceil}(4.67)=5$, $\text{count}=3$

$i=2$: $\text{start}=\text{floor}(14/3)=\text{floor}(4.67)=4$, $\text{end}=\text{ceil}(21/3)=\text{ceil}(7)=7$, $\text{count}=3$

Windows may overlap ($i=0$ covers 0-2, $i=1$ covers 2-4: overlap at 2).

This is acceptable for global average pooling tasks.

Use case: force any sequence length to a fixed $L_{out}=1$ (global avg pool) before feeding to a classifier. Common in speaker ID architectures.

9.2 -- adaptiveAvgPool1dForwardFloat

for (b ...) for (ch ...) for (outIdx=0; outIdx<outputLen; outIdx++) {	
windowSlice1d_t w = adaptiveWindow1dAt(outIdx, inputLen, outputLen);	
float sum=0.f;	
for (size_t k=w.start; k<w.end; k++)	
sum += inArr[b*S+ch*T+k];	
outArr[b*S2+ch*T2+outIdx] = sum / (float)w.count;	<-- divide by actual count
}	

9.3 -- adaptiveAvgPool1dBackwardFloat

memset(propLoss->data, 0, totalBytes);	
for (b ...) for (ch ...) for (outIdx=0; outIdx<outputLen; outIdx++) {	
windowSlice1d_t w = adaptiveWindow1dAt(outIdx, inputLen, outputLen);	
float gyVal = gyArr[b*S+ch*T+outIdx];	<-- output gradient
float distributed = gyVal / (float)w.count;	
for (size_t k=w.start; k<w.end; k++)	
gxArr[b*S2+ch*T2+k] += distributed;	
}	

WARNING: AdaptiveAvgPool vs AvgPool Backward

AvgPool backward always divides by kernelSize (fixed K). AdaptiveAvgPool backward divides by w.count (variable, recomputed per window). The adaptive version does not need argmaxArr (like MaxPool) or a fixed stride -- the window is fully determined by (outIdx, L_in, L_out) at any time.

Chapter 10 -- QuantizationLayer.h / QuantizationLayer.c

GAP IDENTIFIED

GAP: The straight-through estimator and dispatchQuantization routing logic were unexplained.

MISSING: why STE is correct for quantization backward, dispatchQuantization type-dispatch, SYM_INT32 requant via conversionMatrix diagonal, same-dtype non-quantize error.

10.1 -- dispatchQuantization: Type-Based Routing

```
static void dispatchQuantization(tensor_t *src,
                                tensor_t *dst) {
    BOOL sameType = (src->quantizationConfig->type ==
                     dst->quantizationConfig->type);
    if (sameType) {
        if (src->quantizationConfig->type == SYM_INT32) {
            /* requantize: different scales, same dtype */
            /* use conversionMatrix diagonal */
            requantizeSYMINT32(src, dst);
        } else {
            /* FLOAT32 -> FLOAT32 is a no-op, should not
             happen */
            logError("same dtype non-quantize");
        }
    } else {
        /* cross-dtype: FLOAT32 <-> SYM_INT32 */
        convertTensor(src, dst);
    }
}
```

CONCEPT: conversionMatrix Diagonal for SYM_INT32 Requantization

SYM_INT32 represents: $\text{real} = \text{mantissa} * \text{scale}$

To requantize from scale_src to scale_dst :

$\text{mantissa_dst} = \text{round}(\text{mantissa_src} * \text{scale_src} / \text{scale_dst})$

The conversionMatrix is a diagonal matrix where:

$\text{conversionMatrix}[i][i] = \text{scale_src} / \text{scale_dst}$

Applying it to the mantissa vector gives the requantized mantissas.

Real-value preserved: $\text{mantissa_dst} * \text{scale_dst}$

$= \text{round}(\text{mantissa_src} * \text{scale_src} / \text{scale_dst}) * \text{scale_dst}$

$\approx \text{mantissa_src} * \text{scale_src} = \text{real value (with rounding error)}$

The diagonal ensures no cross-channel mixing (element i only maps to i).

10.2 -- quantizationForward

<pre>void quantizationForward(layer_t *l, tensor_t *input, tensor_t *output) {</pre>	
<pre>dispatchQuantization(input, output);</pre>	<-- all work done here
<pre>}</pre>	

10.3 -- quantizationBackward: Straight-Through Estimator

The quantization function $Q(x)$ is piecewise-constant -- its true gradient is zero almost everywhere and undefined at discontinuities. The STE replaces the true gradient with 1.

<pre>void quantizationBackward(layer_t *l, tensor_t *input,</pre>	
<pre>tensor_t *loss, tensor_t *propLoss) {</pre>	
<pre>/* STE: pass loss through quantization unchanged *\</pre>	
<pre>dispatchQuantization(loss, propLoss);</pre>	<-- quantize the GRADIENT
<pre>}</pre>	

CONCEPT: Straight-Through Estimator (STE): Theory

$Q(x) = \text{round}(x/s) * s$ (quantization)

True gradient: $dQ/dx = 0$ everywhere (floor/round is flat between integers).

If we used the true gradient: every weight update = 0 -> no learning.

STE (Hinton 2012, Bengio 2013): pretend $dQ/dx = 1$.

$$dL/dx \approx dL/dQ(x) * 1 = dL/dQ(x)$$

Intuition: quantization is a nearly-identity function for small step sizes s . The error $Q(x)-x$ is bounded by $s/2$. So treating it as identity in backward is a small approximation error.

In ODT: the gradient tensor (loss) is itself quantized via `dispatchQuantization` to match the dtype/scale of `propLoss`, but the VALUE is passed through unchanged.

STE is the standard approach for quantization-aware training (QAT) in all major frameworks (PyTorch FakeQuant, TensorFlow TFLite training, etc.).

Chapter 11 -- Conv1d.h / Conv1d.c

GAP IDENTIFIED

GAP: Grouped convolution indexing and the weight/bias gradient accumulation were not covered.

MISSING: group decomposition (inChPerGroup, outChPerGroup), windowSlice1dAt in conv, weight grad accumulation loop order, bias grad sum across batch and length.

11.1 -- initConv1dConfigWithWeightsAndBias

conv1dConfig_t	
*initConv1dConfigWithWeightsAndBias(	
size_t inChannels, size_t outChannels, size_t	
kernelSize,	
size_t stride, size_t padding, size_t groups,	
parameter_t *weights, parameter_t *bias) {	
c->inChPerGroup = inChannels / groups;	<-- input channels per group
c->outChPerGroup = outChannels / groups;	<-- output channels per group
/* weights shape: [outCh, inChPerGroup,	
kernelSize] */	
return c;	
}	

CONCEPT: Grouped Convolution

groups=1: Standard conv. Every output channel sees all input channels.

groups=inCh=outCh: Depthwise conv. Channel i output only sees channel i input.

groups=G: Input split into G groups of inCh/G channels each.

Each output group (outCh/G channels) connects to one input group.

Memory layout for weights: [outCh, inChPerGroup, K]

Group g uses weights[g*outChPerGroup .. (g+1)*outChPerGroup-1]

and inputs[g*inChPerGroup .. (g+1)*inChPerGroup-1]

RigL: Sparse mask shape matches weights shape. Each (oc, ic, k) corresponds to one mask bit. Pruning zeros mask bits; growing sets them.

11.2 -- conv1dKernelFloat32: Forward Computation

for (b=0; b<batch; b++)	
for (g=0; g<groups; g++)	<-- group loop
for (oc=0; oc<outChPerGroup; oc++) {	<-- output channel within group
size_t ocAbs = g*outChPerGroup + oc;	<-- absolute output channel
for (outPos=0; outPos<outLen; outPos++) {	

windowSlice1d_t w = windowSlice1dAt(outPos, stride, padding, K, inLen);	
float acc = 0.f;	
for (ic=0; ic<inChPerGroup; ic++) {	<-- input channels in group
size_t icAbs = g*inChPerGroup + ic;	
for (k=w.kernelStart; k<K; k++) {	<-- kernel position
size_t inPos = w.start + k - w.kernelStart;	
acc += inArr[b][icAbs][inPos] * wArr[ocAbs][ic][k];	
}	
}	
if (bias) acc += biasArr[ocAbs];	
outArr[b][ocAbs][outPos] = acc;	
}	
}	

11.3 -- conv1dCalcWeightGradsFloat32

Weight gradient: $dL/dW[oc,ic,k] = \text{sum over (batch, outPos) of } input[b][icAbs][inPos] * lossGrad[b][ocAbs][outPos]$.

for (b ...) for (g ...) for (oc ...) for (outPos ...) {	
float gyVal = gyArr[b][g*outChPerGroup+oc][outPos];	<-- output gradient
for (ic ...) for (k ...) {	
size_t inPos = w.start + k - w.kernelStart;	
gwArr[g*outChPerGroup+oc][ic][k] += inArr[b][g*inChPerGroup+ic][inPos] * gyVal;	<-- accumulate
}	
}	

WARNING: Weight Grad Accumulation (+= not =)

Weight grads are SUMMED over the batch dimension and all output positions. The += accumulation pattern is essential: if we used =, each (b, outPos) pair would overwrite the previous value, giving only the last iteration's gradient. gwArr must be zero-initialized before the loop (done by arena allocator).

11.4 -- conv1dCalcBiasGradsFloat32

for (b ...) for (g ...) for (oc ...) {	
for (outPos ...) {	
gbArr[g*outChPerGroup+oc] += gyArr[b][g*outChPerGroup+oc][outPos];	
}	
}	

CONCEPT: Bias Grad = Sum Over Batch and Length

Bias b[oc] is broadcast across all (batch, position) pairs.

$$dL/db[oc] = \text{sum over } (b, \text{outPos}) \text{ of } dL/dout[b][oc][\text{outPos}]$$

This follows from the chain rule: the bias contributes to every output position, so every output gradient flows back to it.

Sum = total gradient across all positions that used this bias value.

Chapter 12 -- Conv1dTransposed.h / Conv1dTransposed.c

GAP IDENTIFIED

GAP: Transposed convolution scatter-based forward and the outputPadding constraint were unexplained.

MISSING: why transposed conv is NOT the inverse of Conv1d, scatter pattern (outBase=inPos*stride), VALID-only padding constraint, outputPadding purpose.

12.1 -- initConv1dTransposedConfigWithWeightsAndBias

conv1dTransposedConfig_t	
*initConv1dTransposedConfigWithWeightsAndBias(	
..., size_t padding, size_t outputPadding, ...) {	
if (padding != 0) { logError("only VALID	<-- VALID only
(padding=0) supported"); }	
if (outputPadding >= stride) {	
logError("outputPadding must be < stride"); }	
/* store config */	
}	

CONCEPT: VALID Padding Only and outputPadding

Conv1dTransposed in the ODT library supports only VALID padding (padding=0).

This means no implicit padding is added to the input before the transpose op.

outputPadding: When stride > 1, multiple input sizes can produce the same output size after a forward Conv1d. To uniquely determine the output size of the transposed conv, outputPadding adds [0, outputPadding) extra zeros at the end of the output.

Constraint: $0 \leq \text{outputPadding} < \text{stride}$.

Output size formula (VALID, no padding):

$\text{outLen} = (\text{inLen} - 1) * \text{stride} + \text{kernelSize} + \text{outputPadding}$

12.2 -- conv1dTransposedForwardFloat: Scatter Pattern

Transposed conv is the gradient of Conv1d w.r.t. its input. It works by scattering each input value across the output via the kernel.

/* zero-initialize output */	
memset(out->data, 0, totalBytes);	
for (b ...) for (g ...) for (ic ...) for (inPos ...)	<-- loop over INPUT positions
{	
size_t icAbs = g*inChPerGroup + ic;	
size_t outBase = inPos * stride;	<-- SCATTER: base output position
for (oc ...) {	<-- output channel

size_t ocAbs = g*outChPerGroup + oc;	
float inVal = inArr[b][icAbs][inPos];	
for (k=0; k<K; k++) {	
outArr[b][ocAbs][outBase+k] += inVal * wArr[ocAbs][ic][k];	<-- scatter
}	
}	
}	

CONCEPT: Scatter vs Gather: Conv1d vs Conv1dTransposed

Conv1d (forward): GATHER

for each OUTPUT position: gather a window of inputs, dot with kernel

output[b][oc][outPos] = sum_ic sum_k input[b][ic][outPos*stride+k] * W[oc][ic][k]

Conv1dTransposed (forward): SCATTER

for each INPUT position: scatter its contribution across output positions

output[b][oc][inPos*stride + k] += input[b][ic][inPos] * W[oc][ic][k]

The scatter pattern is why the output must be zero-initialized: multiple input positions contribute to the same output position (overlap at +stride gaps).

Transposed conv is NOT the mathematical inverse of Conv1d. It IS the transpose of the Conv1d linear operator (hence the name), which makes it exactly what is needed to backpropagate gradients through Conv1d.

12.3 -- conv1dTransposedCalcWeightGradsFloat32

Weight gradients for transposed conv: sum over (batch, inPos) of input[b][ic][inPos] * lossGrad[b][oc][inPos*stride+k].

for (b ...) for (g ...) for (ic ...) for (inPos ...)	
{	
for (oc ...) for (k=0; k<K; k++) {	
size_t outBase = inPos * stride;	
gwArr[g*outChPerGroup+oc][ic][k] +=	
inArr[b][g*inChPerGroup+ic][inPos] *	
gyArr[b][g*outChPerGroup+oc][outBase+k];	
}	
}	

WARNING: Transposed Conv Weight Grads: Gather from Loss

In Conv1d, weight grads gather from the INPUT at the conv window position. In Conv1dTransposed, weight grads gather from the LOSS at position inPos*stride+k. The inPos*stride pattern mirrors the scatter pattern from the forward pass: each input position scattered to outBase=inPos*stride, so gradient flows back from those same output positions.

Glossary

AdaptiveAvgPool1d

Pooling layer that maps any input length to a fixed output length by computing variable-size window averages.

Arena allocator

reserveMemory(): bump-pointer allocator; O(1) alloc; no fragmentation; no free.

argmaxArr

Array storing which input position had the max value in each MaxPool window. Used in backward scatter.

AvgPool1d

Averages a sliding window; distributes gradient equally in backward.

bernoulliFillMask

Fills a BOOL tensor with Bernoulli samples: each bit=1 with probability keep_prob.

BOOL tensor

Tensor storing 1 bit per weight; accessed via tensorBoolGet/Set. Used for RigL sparsity masks.

calcElementIndexByIndices

Like calcTensorIndexByIndices but applies orderOfDimensions permutation first.

Conv1d

1D grouped convolution. Forward=gather; weight grads accumulate over batch and output positions.

Conv1dTransposed

Transpose of Conv1d linear operator. Forward=scatter; output must be zero-initialized.

conversionMatrix

Diagonal matrix used in SYM_INT32 requantization to scale mantissas between quantization configs.

count_include_pad

AvgPool1d always divides by K (kernelSize), even at boundaries where zeros were padded.

Dropout

Bernoulli mask + inverted scaling ($1/(1-p)$). Scale-fold for SYM_INT32. training=false is safe default.

eps

Small float added INSIDE sqrt in LayerNorm: $\sqrt{\text{var}+\text{eps}}$. Prevents division by zero.

Flatten

Collapses all post-batch dims into one. Forward/backward are memcpy; scale is copied for SYM_INT32.

initLayer

Wires a layerConfig_t pointer into a layer_t and sets layerType. Does NOT allocate config.

inverted dropout

Scale kept units by $1/(1-p)$ during training so inference needs no scaling.

Jacobian-free backward

Softmax backward using dot product trick: $dLdx[i] = s[i]*(dLds[i]-dot(s,dLds))$.

layerFunctions[]

Dispatch table indexed by `layerType_t`. Designated initializers [RELU]={f,b,s} ensure correct wiring.

LayerNorm

Normalizes over groups. Two-pass stats. eps inside sqrt. Affine gamma/beta per group.

layerNormPhysOffset

Maps (batch, group, elem) to byte offset using `calcElementIndexByIndices` for transposed safety.

layerType_t

Enum tagging the variant type of a `layer_t`.

MaxPool1d

Takes max in sliding window. Stores argmax in forward for scatter-based backward.

orderOfDimensions

Dimension permutation enabling zero-copy transpose in `tensor_t`.

outputPadding

Extra zeros appended to `Conv1dTransposed` output to disambiguate output size. Must be $< \text{stride}$.

parameter_t

Bundles a weight tensor + its gradient tensor (same shape).

quantizationCalcOutputShape

Same shape as input; dispatches quantization dtype.

QuantizationLayer

Forward: `dispatchQuantization`. Backward: STE (pass loss through same dispatch).

ReLU

Forward: $\max(0, x)$. Backward: gradient gate (zero where $\text{forwardInput} \leq 0$). Scale-transparent.

RigL

Sparse training: prune lowest $|w|$, grow highest $|\text{grad}|$ every T steps. Mask stored as `BOOL` tensor.

scale-folding

Folding a scale factor into quantization config scale rather than multiplying every element.

scatter

Writing to output positions indexed by a formula (e.g., $\text{outBase} = \text{inPos} * \text{stride}$). Used in `MaxPool` backward and `Conv1dTransposed` forward.

Softmax

3-pass stable: find max, $\exp(x-\text{max})+\text{sum}$, normalize. Backward: Jacobian-free $O(N)$ formula.

STE

Straight-Through Estimator: approximate $dQ/dx=1$ for quantization backward to allow gradient flow.

SYM_INT32

Symmetric fixed-point: $\text{real} = \text{mantissa} * \text{scale}$. $q\text{Bits}=8$ storage, $q\text{MaxBits}=16$ training.

VLA

Variable-Length Array: C99 stack allocation of runtime-sized buffer. Risk of stack overflow on 320KB MCU.

windowSlice1d_t

Struct {start, end, kernelStart, count} describing one conv/pool window into input.

zero-copy transpose

`transposeTensor()` swaps entries in `orderOfDimensions`; no data moved.